

SCALE: State-Calibrated Latent Embeddings for JEPA Planning in the Right Geometry

Jiaming Hu^{1,2*} Yan Zheng² Tian Wang² Florian Dubost²
 Alejandro Mottini² Junze Liu² Arvind Srinivasan² Kai Zhong²
 Kun Qian² Sharon Gao² Qingjun Cui²
¹Boston University ²Unity Technologies
 {jiaming.hu, yan.zheng, tianwang, florian.dubost,
 alejandro.mottini, junze.liu, arvind.srinivasan, kai.zhong,
 kun.qian, sharon.gao, qingjun.cui}@unity3d.com

Abstract

Joint-embedding predictive world models plan by scoring predicted terminal embeddings against a goal embedding using a cost defined on the representation itself. Two prominent strategies for obtaining non-collapsed representations are to inherit a pretrained feature space, as in DINO-WM, and to learn an embedding end to end with anti-collapse regularization, as in LeWorldModel (LeWM) with SIGReg. These strategies show complementary strengths across tasks. Although task-relevant state is decodable from the full embeddings of both models, DINO-WM’s leading principal components usually retain substantially more state information than LeWM’s. Because Euclidean planning costs are dominated by high-variance directions, this difference affects how strongly state can influence candidate selection. We propose SCALE (State-CALibrated Latent Embeddings) to give the end-to-end LeWM representation the favorable geometric property observed in DINO-WM. SCALE induces this property by correlating sampled pairwise latent distances with distances in a standardized task-relevant state space, without replacing LeWM’s learned encoder. Across five tasks, three planning solvers, and five compute budgets, SCALE improves every task–solver average over LeWM. A latent-to-state regression control matches or exceeds SCALE’s full-embedding decodability yet leaves latent–state distance alignment essentially unchanged and yields less consistent planning gains. SCALE adds a single lightweight training-time regularizer and no planning-time overhead. These results show that planning depends not only on whether task-relevant information is present, but also on whether it shapes the geometry consumed by the planner.

1 Introduction

World models support control by predicting the consequences of candidate actions before they are executed [Ha and Schmidhuber, 2018, Hafner et al., 2020]. Reconstruction-based visual world models predict future observations in pixel space, but high-fidelity reconstruction can devote capacity to appearance details that are irrelevant to control. Latent-predictive world models instead forecast representations of future observations, offering a compact space for prediction and planning [Hafner et al., 2019, Zhou et al., 2024, Sobal et al., 2025, Maes et al., 2026]. In JEPA-based variants [As-sran et al., 2023], an encoder maps an observation to a latent, an action-conditioned predictor rolls that latent forward, and a planner scores each candidate action sequence by a cost evaluated on its predicted terminal embedding and a goal embedding. The planner never inspects the representation directly: this cost is its entire interface to what the model has learned.

*Correspondence to jh7453@bu.edu. This work was completed while Jiaming Hu was an intern at Unity Technologies.

LeWorldModel (LeWM) provides a concrete instance of this setup. It jointly learns an encoder and an action-conditioned dynamics predictor, and evaluates imagined futures through distances between predicted terminal and goal embeddings. To prevent the collapsed representations admitted by end-to-end joint-embedding training, LeWM uses SIGReg, which encourages a globally well-spread, approximately isotropic embedding distribution. This regularization, however, does not determine which factors of variation should most strongly shape the distances used for planning. A representation may therefore predict accurately, remain non-collapsed, and retain task-relevant state information, while that information has little influence on the planner’s cost. This raises a central question: what makes a latent geometry useful for planning?

Comparing LeWM with DINO-WM reveals a useful contrast. Both representations retain substantial task-relevant state information, yet they expose that information differently through the geometry used for planning. In DINO-WM, variations in task state are more strongly reflected in latent distances, whereas in LeWM the same information can remain readily decodable without strongly shaping those distances. This suggests that the relevant distinction is not simply whether task-relevant information is encoded, but whether its variation is expressed in the metric through which the planner interacts with the representation.

This distinction matters because the planner has no learned decoder with which to recover and reweight latent information; it acts directly through the planning cost. Under the Euclidean cost used by LeWM and DINO-WM, directions with larger variation contribute more, on average, to latent distances. Task-relevant information concentrated in weakly varying directions can therefore remain easy to decode while having little effect on candidate ranking. Decodability and geometric influence are thus distinct properties of a representation.

Motivated by this contrast, we seek to transfer the favorable geometric property observed in DINO-WM without inheriting its computational cost. In our implementation, DINO-WM requires approximately $2.5\times$ the training time and nearly $50\times$ the planning time of LeWM under matched hardware and planner settings. We therefore propose SCALE (State-CALibrated Latent Embeddings), a lightweight training-time regularizer that augments LeWM’s existing objective. SCALE induces the desired geometry by correlating pairwise latent distances with distances in task-relevant state space, without extra planning cost.

Empirically, this geometric intervention translates into consistently better planning: SCALE outperforms LeWM on all 15 task–solver averages, with gains persisting across compute budgets. To test whether these improvements can be explained simply by stronger state encoding, we compare against an auxiliary latent-to-state regression objective. Although this control matches or exceeds SCALE’s full-embedding decodability on two tasks, it yields less consistent planning improvements. Together, these results support the distinction between encoding task-relevant information and making that information geometrically influential.

Our contributions are:

- We identify *metric leverage*—the weight a latent direction carries in the planner-facing cost—and show that under the squared Euclidean metric it is exactly the direction’s variance (Sec. 4).
- We introduce SCALE, a state-calibration regularizer that reshapes latent-space geometry so pairwise latent distances reflect task-relevant state differences. (Sec. 5).
- We empirically verify SCALE with extensive experiments: across five environments, three planning solvers, and five compute budgets, it improves all 15 task–solver averages over LeWM, while a state-regression control (Aux) with identical supervision fails to match it (Sec. 6).

2 Related Work

Latent-predictive world models. World models support control through several mechanisms, including learning policies from imagined trajectories [Ha and Schmidhuber, 2018, Hafner et al., 2020] and optimizing action sequences online using model rollouts [Chua et al., 2018, Nagabandi et al., 2018, Hansen et al., 2024]. Reconstruction-free approaches predict future latent representations rather than pixels. DINO-WM predicts pretrained DINOv2 patch features and plans by optimizing action sequences [Oquab et al., 2024, Zhou et al., 2024]; PLDM jointly trains an encoder and

latent dynamics model with latent-prediction, VICReg-inspired anti-collapse, and inverse-dynamics objectives [Bardes et al., 2022, Sobal et al., 2025]; and LeWM combines next-embedding prediction with SIGReg [Maes et al., 2026]. Concurrent LpWM replaces LeWM’s dense Gaussian-matched representation with non-negative sparse codes to reduce predictor complexity and encourage mode-factored dynamics [Kuang et al., 2026]. These models differ in how they obtain non-collapsed representations, but none explicitly trains the latent-space planning cost to reflect task-relevant differences. SCALE retains LeWM’s prediction and SIGReg objectives while adding state supervision for this purpose.

Supervising latent geometry. Several approaches add supervision to make JEPA representations more useful for planning. Value-guided JEPA aligns latent distances with goal-conditioned value [Destrade et al., 2025]; SCALE instead aligns them with logged simulator state, requiring no reward, value estimate, or policy. Two concurrent methods use such privileged state for grounding: PhyLatent [Zeng et al., 2026] and PSG-JEPA [Yan et al., 2026] train auxiliary heads to predict physical state or state changes from latents. Notably, PhyLatent diagnoses physically distant pairs that remain latent-close in LeWM—independent evidence for our premise, which SCALE turns from a diagnostic into a differentiable objective. Head-based grounding certifies that state is *decodable*; because a head can be an arbitrary nonlinear map, it leaves the metric unconstrained. Our Aux control implements exactly this form of supervision, and SCALE constrains the pairwise distance directly.

3 Preliminaries

3.1 Latent-space planning

Let o_t and $\mathbf{a}_t$ denote the observation and action at time t . The encoder E_θ produces $\mathbf{z}_t = E_\theta(o_t)$, and the action-conditioned predictor P_ψ predicts $\hat{\mathbf{z}}_{t+1} = P_\psi(\mathbf{z}_t, \mathbf{a}_t)$. At test time, given a candidate action sequence $\mathbf{a}_{0:H-1}$ and current observation o_0 , the model sets $\hat{\mathbf{z}}_0 = E_\theta(o_0)$ and rolls out

$$\hat{\mathbf{z}}_{t+1} = P_\psi(\hat{\mathbf{z}}_t, \mathbf{a}_t), \quad t = 0, \dots, H-1. \quad (1)$$

For a goal observation o_g with embedding $\mathbf{z}_g = E_\theta(o_g)$, the planner ranks candidates by a scalar cost

$$J(\mathbf{a}_{0:H-1}) = C(\hat{\mathbf{z}}_H(\mathbf{a}_{0:H-1}), \mathbf{z}_g), \quad (2)$$

computed entirely from the learned representation. In LeWM and DINO-WM this cost is the squared Euclidean distance

$$J(\mathbf{a}_{0:H-1}) = \|\hat{\mathbf{z}}_H(\mathbf{a}_{0:H-1}) - \mathbf{z}_g\|_2^2, \quad (3)$$

Because the solver interacts with the representation only through J , its decisions depend entirely on the distance geometry the encoder has learned.

3.2 LeWM training objective

For a transition $(o_t, \mathbf{a}_t, o_{t+1})$, the encoder and predictor are trained with the one-step latent-prediction loss

$$\mathcal{L}_{\text{pred}} = \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2^2, \quad (4)$$

where $\mathbf{z}_{t+1} = E_\theta(o_{t+1})$ is the target embedding. This loss alone admits collapsed solutions in which the encoder maps every observation to the same point. For a batch of N encoder outputs $Z = \{\mathbf{z}_i\}_{i=1}^N$, SIGReg samples random unit directions in latent space, projects Z onto each, and penalizes deviations of the projected empirical distributions from a univariate standard normal [Balestriero and LeCun, 2025, Maes et al., 2026]. Applied across many directions, this drives the embedding distribution toward an isotropic Gaussian and maintains global spread. LeWM therefore optimizes

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \text{SIGReg}(Z). \quad (5)$$

Neither term specifies which task-relevant differences should dominate pairwise latent distances. A representation can satisfy both while organizing its geometry around variation that has nothing to do with the task.

4 Where Task-Relevant State Lives

4.1 A motivating distance alignment contrast

The planner of Sec. 3.1 interacts with the representation only through the scalar cost of Eq. (3): whatever a representation encodes, the planner sees it only insofar as it registers in latent distance. We therefore quantify the contrast sketched in the introduction directly at this interface, computing the Spearman rank correlation between $\|\mathbf{z}_i - \mathbf{z}_j\|_2^2$ and $\|\mathbf{q}_i - \mathbf{q}_j\|_2^2$ over held-out trajectory frame pairs, where $\mathbf{q}$ denotes the task-relevant state variables logged alongside each trajectory (formally specified in Sec. 5.1). For the sampled frame pairs, we rank the latent distances and state distances separately. Spearman ρ measures the agreement between these two rankings: a high value means that pairs farther apart in task state also tend to be farther apart in latent space. This is directly relevant because the planner selects actions by ranking candidates according to latent cost. Ties are assigned average ranks.

Table 1: Latent–state distance rank alignment (Spearman ρ) of the two reference world models over held-out frame pairs. Both models decode most of the same state accurately from their full embeddings (Appendix B); their metrics reflect it to very different degrees.

Task	LeWM	DINO-WM
Push-T	.13	.70
Reacher	.18	.46
Cube	.00	.46
Two-Room	.41	.85
PointMaze	.52	.50

Table 1 reveals a clear contrast. DINO-WM’s latent distances track state differences much more closely than LeWM’s on most tasks. The metric the planner will consume can be uninformative about task-relevant differences, even though probes still recover most of that state accurately from the full embedding (Appendix B). We next ask how such a dissociation between decodability and metric alignment can arise.

4.2 Metric leverage: which directions can influence the cost

Let μ be the mean of the encoded distribution and $\{(\lambda_j, \mathbf{u}_j)\}_{j=1}^D$ its eigenvalue–eigenvector pairs, ordered by decreasing λ_j . Writing a centered embedding in this basis,

$$\mathbf{z} - \mu = \sum_{j=1}^D c_j \mathbf{u}_j, \quad \text{Var}(c_j) = \lambda_j, \quad (6)$$

the cost of Eq. (3) separates into independent per-direction contributions,

$$\|\mathbf{z}_i - \mathbf{z}_g\|_2^2 = \sum_{j=1}^D (c_{ij} - c_{gj})^2. \quad (7)$$

For a pair drawn independently from the encoded distribution, the expected contribution of direction j is

$$\mathbb{E}[(c_{ij} - c_{gj})^2] = 2\lambda_j. \quad (8)$$

The scale at which a direction participates in the cost is thus set by its eigenvalue. Under the squared Euclidean cost, we accordingly define the *metric leverage* of a direction as its expected contribution to the cost, which by Eq. (8) equals $2\lambda_j$: the weight it carries in the scalar that orders candidate action sequences.

This resolves the dissociation of Table 1. Suppose a task-relevant variable $\mathbf{q}$ is encoded entirely in directions with small λ_j . A probe with sufficient capacity recovers $\mathbf{q}$ from the full embedding with high accuracy, since decodability does not depend on the variance of the directions used. The planner, however, applies no learned head and cannot reweight directions; under Eq. (7) those directions

contribute negligibly to J , and candidates differing substantially in $\mathbf{q}$ may receive near-identical costs. In short,

$$\mathbf{q} \text{ is decodable from } \mathbf{z} \not\Rightarrow \mathbf{q} \text{ strongly affects } C(\mathbf{z}, \mathbf{z}_g).$$

A representation in LeWM’s position—state decodable, alignment near zero—is one that stores task-relevant variation in directions with little metric leverage.

Hypothesis. Together, Table 1 and Sec. 4.2 suggest that planning performance depends not merely on whether task-relevant state is represented, but on whether it registers in latent distance. We therefore seek an objective that gives task-relevant variation direct influence over the metric.

5 Method

5.1 State-Calibrated Latent Embeddings

SCALE directly reshapes the pairwise latent distance used for planning. During training, each observation o_i is paired with a simulator state $\mathbf{q}_i^{\text{raw}}$. We construct the task-relevant state $\mathbf{q}_i$ by selecting task-specific dimensions, representing angles with sine–cosine pairs, excluding velocities, and standardizing each component using fixed training-set statistics. This standardized state defines the target geometry and is used only as a detached training target; the encoder and planner remain image-only at test time. Because SCALE operates on pairwise distances, it requires neither equal dimensionality nor coordinate-wise correspondence between the latent and state spaces. The task-specific state selections are detailed in Appendix B.

Consider a minibatch of B sub-trajectories, each containing T frames. We flatten its $N = BT$ frames into $\{(\mathbf{z}_n, \mathbf{q}_n, b_n, e_n)\}_{n=1}^N$, where $\mathbf{z}_n \in \mathbb{R}^D$ is an encoder output, $\mathbf{q}_n \in \mathbb{R}^{d_q}$ is its standardized task-relevant state, b_n identifies its sub-trajectory, and e_n its source episode. At each step, we sample a set $\mathcal{P} = \{(i_k, j_k)\}_{k=1}^K$ of distinct frame pairs, with half drawn within sub-trajectories ($b_{i_k} = b_{j_k}$) and half across episodes ($e_{i_k} \neq e_{j_k}$). This stratification supplies both local and global geometric comparisons. We then define the latent and state distance profiles

$$x_k = \|\mathbf{z}_{i_k} - \mathbf{z}_{j_k}\|_2^2, \quad y_k = \|\mathbf{q}_{i_k} - \mathbf{q}_{j_k}\|_2^2. \quad (9)$$

Let $\bar{x}$ and s_x be the mean and standard deviation of $\{x_k\}_{k=1}^K$; $\bar{y}$ and s_y are defined analogously. With a numerical constant $\varepsilon > 0$, the standardized profiles and correlation loss are

$$\tilde{x}_k = \frac{x_k - \bar{x}}{s_x + \varepsilon}, \quad \tilde{y}_k = \frac{y_k - \bar{y}}{s_y + \varepsilon}, \quad \mathcal{L}_{\text{corr}} = 1 - \frac{1}{K} \sum_{k=1}^K \tilde{x}_k \tilde{y}_k. \quad (10)$$

Component-wise normalization of $\mathbf{q}_n$ makes the selected state variables comparable, while normalization of the two distance profiles makes $\mathcal{L}_{\text{corr}}$ depend on relative distance structure rather than global scale. In the ideal case, x_k and y_k are perfectly positively linearly related, their standardized profiles coincide, and $\mathcal{L}_{\text{corr}} = 0$: larger state differences then produce proportionally larger latent distances.

Therefore, the complete objective is

$$\mathcal{L}_{\text{SCALE}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \text{SIGReg}(Z) + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}}, \quad (11)$$

where the three terms play complementary roles: $\mathcal{L}_{\text{pred}}$ trains action-conditioned latent prediction, SIGReg prevents collapse and maintains global embedding spread, and $\mathcal{L}_{\text{corr}}$ redistributes information within that spread so task-relevant variation carries greater metric leverage. The complete $\mathcal{L}_{\text{SCALE}}$ updates the encoder E_θ , whereas the predictor P_ψ is updated only by $\mathcal{L}_{\text{pred}}$. Training pseudocode is provided in Appendix A.1.

5.2 Latent-to-state regression as a mechanistic control

To separate the two properties distinguished in Sec. 4.2, we train a control that optimizes decodability while leaving the metric unconstrained. The Aux baseline attaches a training-only head h_ϕ to the encoder output and minimizes

$$\mathcal{L}_{\text{aux}} = \|h_\phi(\mathbf{z}_n) - \mathbf{q}_n\|_2^2 \quad (12)$$

Table 2: iCEM success rate (%) across tasks and compute tiers. Each entry reports mean $\pm$ SD over six evaluation sets; the final column averages T1–T5. DINO-WM is included as a pretrained-feature reference. Bold marks the best method per column. Appendix C reports complete results for all planning solvers.

Task	Method	T1	T2	T3	T4	T5	Mean
Push-T	LeWM	86.2 $\pm$ 3.3	80.7 $\pm$ 2.2	73.5 $\pm$ 6.5	53.5 $\pm$ 5.7	25.7 $\pm$ 5.4	63.9
	SCALE	89.7 $\pm$ 2.0	87.0 $\pm$ 2.5	79.0 $\pm$ 3.9	59.8 $\pm$ 4.7	25.5 $\pm$ 5.8	68.2
	Aux.	91.3$\pm$1.2	87.7$\pm$1.8	80.3$\pm$4.0	60.5$\pm$6.0	26.7$\pm$4.6	69.3
	DINO-WM	70.5 $\pm$ 4.2	65.2 $\pm$ 4.2	58.5 $\pm$ 5.2	39.8 $\pm$ 10.0	22.5 $\pm$ 4.3	51.3
Reacher	LeWM	85.5 $\pm$ 2.4	81.8 $\pm$ 3.1	83.3 $\pm$ 3.5	77.8 $\pm$ 6.1	61.7 $\pm$ 2.9	78.0
	SCALE	86.7$\pm$3.6	86.2 $\pm$ 2.9	86.5 $\pm$ 3.4	82.3$\pm$3.2	64.0$\pm$2.5	81.1
	Aux.	84.3 $\pm$ 4.6	87.0$\pm$2.8	86.8$\pm$3.5	81.2 $\pm$ 3.8	61.5 $\pm$ 4.1	80.2
	DINO-WM	79.0 $\pm$ 4.6	77.5 $\pm$ 5.6	79.7 $\pm$ 3.1	75.5 $\pm$ 5.0	52.3 $\pm$ 6.0	72.8
Cube	LeWM	70.7 $\pm$ 5.3	68.0 $\pm$ 3.9	69.7 $\pm$ 3.6	62.7 $\pm$ 4.7	55.2 $\pm$ 5.7	65.3
	SCALE	76.7$\pm$3.9	76.5$\pm$4.5	74.8$\pm$3.5	64.8$\pm$3.7	59.7$\pm$4.5	70.5
	Aux.	74.5 $\pm$ 3.2	73.3 $\pm$ 4.5	73.0 $\pm$ 3.1	64.5 $\pm$ 3.3	57.7 $\pm$ 3.8	68.6
	DINO-WM	70.8 $\pm$ 2.6	67.3 $\pm$ 1.5	66.2 $\pm$ 1.7	57.3 $\pm$ 4.5	51.7 $\pm$ 3.8	62.7
Two-Room	LeWM	91.3 $\pm$ 1.2	93.3 $\pm$ 1.2	89.2 $\pm$ 2.5	77.5 $\pm$ 1.4	63.2 $\pm$ 3.0	82.9
	SCALE	100.0$\pm$0.0	99.8 $\pm$ 0.4	97.3 $\pm$ 1.4	90.5 $\pm$ 1.4	76.7 $\pm$ 1.9	92.9
	Aux.	92.5 $\pm$ 2.7	93.5 $\pm$ 2.1	88.5 $\pm$ 1.9	79.0 $\pm$ 2.4	62.0 $\pm$ 1.8	83.1
	DINO-WM	100.0 $\pm$ 0.0	100.0$\pm$0.0	99.7$\pm$0.5	98.7$\pm$0.5	92.8$\pm$1.7	98.2
PointMaze	LeWM	81.7 $\pm$ 3.9	82.7 $\pm$ 4.4	81.3 $\pm$ 3.6	81.7 $\pm$ 5.1	73.7 $\pm$ 4.7	80.2
	SCALE	87.3 $\pm$ 3.9	82.7 $\pm$ 3.9	84.3 $\pm$ 4.1	83.2 $\pm$ 4.6	79.3 $\pm$ 2.4	83.4
	Aux.	83.8 $\pm$ 4.8	82.8 $\pm$ 3.7	80.5 $\pm$ 5.5	81.2 $\pm$ 2.9	78.3 $\pm$ 5.5	81.3
	DINO-WM	94.2$\pm$1.7	93.7$\pm$2.3	92.8$\pm$2.3	90.8$\pm$1.7	86.2$\pm$1.9	91.5

in place of $\mathcal{L}_{\text{corr}}$, using the same selected and standardized state, the same architecture, and the same training protocol. Because h_ϕ may be an arbitrary nonlinear map, Eq. (12) is satisfied by placing $\mathbf{q}$ anywhere in the representation, including directions with negligible metric leverage. Aux therefore poses a sharp question: if making state easier to decode were sufficient, it should match SCALE.

6 Experiments

6.1 Setup

We evaluate LeWM, SCALE, and the latent-to-state regression control on Push-T [Chi et al., 2023], DeepMind Control Reacher [Tassa et al., 2018], OGBench-Cube [Park et al., 2025], Two-Room [Sobal et al., 2025], and PointMaze [Fu et al., 2020, Zhou et al., 2024]. SCALE and the regression control use the same standardized task-relevant state; its task-specific components and selection rationale are detailed in Appendix B. All three configurations share the LeWM architecture and training protocol; only the added supervision differs. Exact architectures and training hyperparameters are given in Appendix A.

We follow the LeWM goal-conditioned planning protocol and evaluate CEM [de Boer et al., 2005], iCEM [Pinneri et al., 2021], and MPPI [Williams et al., 2017], with DINO-WM as a pretrained-feature reference. We report success rates; planner budgets, horizons, temperature selection, and evaluation-set construction are detailed in Appendix A.

6.2 Planning performance

SCALE improves LeWM on every task–solver average (15/15). Under iCEM (Table 2), its gains averaged across budgets are 4.3, 3.1, 5.2, 10.0, and 3.1 percentage points on Push-T, Reacher, Cube, Two-Room, and PointMaze, respectively. The same improvement holds under CEM and MPPI and across a $300\times$ range of rollout budgets (Appendix C), showing that SCALE improves the planner-facing cost rather than one particular search procedure.

The regression control isolates why SCALE is more consistent. Full-embedding probes confirm that Aux successfully encodes the supervised state, even exceeding SCALE’s decodability on Push-T and Cube (Appendix B). Yet Aux improves only 13 of 15 task–solver averages and trails SCALE on four of five tasks; Push-T is the exception. The planning gap therefore cannot just be explained by whether state information is present, but by whether that information shapes latent geometry.

The comparison with DINO-WM further highlights this consistency. DINO-WM is significantly better than LeWM on Two-Room and PointMaze, significantly worse on Push-T ($p = .031$), and not significantly different on Reacher or Cube. SCALE, in contrast, never falls below LeWM on any task-solver pair. It thus learns a consistently useful planner-facing geometry without inheriting the task dependence of a fixed pretrained encoder.

6.3 How SCALE reorganizes the latent geometry

The planning results establish that SCALE improves an end-to-end latent world model, but do not by themselves reveal what changes inside the representation. We therefore trace the effect of SCALE through the geometry itself. We first examine how variance is distributed across latent directions, then identify what information occupies the directions whose variance increases, and finally test whether this reorganization survives model rollout strongly enough to affect the candidate selection used for planning.

SCALE increases variance in the dominant latent directions. We begin with the PCA spectrum of each frozen representation. Figure 1 shows that SCALE consistently raises the variance carried by the leading principal directions relative to LeWM and Aux on most tasks. DINO-WM also exhibits a strongly weighted spectral head, although its much higher dimensionality produces a qualitatively different long-tail spectrum.

Under the squared Euclidean planning cost, this change has a direct geometric interpretation. As shown in Sec. 4.2, a principal direction with eigenvalue λ_j contributes $2\lambda_j$ in expectation to the squared distance between independently sampled embeddings. Increasing variance near the head of the spectrum therefore gives these directions greater average leverage over the cost consumed by the planner. The spectrum alone, however, does not tell us what factors of variation have acquired that leverage.

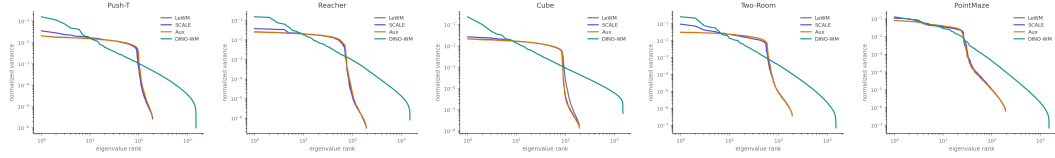


Figure 1: Normalized PCA eigenvalue spectra of the frozen embeddings. SCALE places more variance in the leading principal directions than LeWM and Aux on most tasks. DINO-WM also exhibits a strongly weighted spectral head, while its substantially larger representation produces a much longer spectral tail.

The additional dominant variation is task relevant. We next ask what the high-variance directions actually encode. For each frozen representation, we regress the selected task-relevant state $\mathbf{q}$ using only its leading k principal components. Figure 2 shows that the leading components of SCALE are substantially more predictive of $\mathbf{q}$ than those of LeWM. The separation from Aux is especially informative: although Aux can recover $\mathbf{q}$ accurately from the full embedding, substantially less of that information appears in its leading components.

Together with the spectral analysis, this identifies what SCALE changes. It does not merely increase variance in arbitrary latent directions; the directions receiving greater metric leverage become more strongly associated with task-relevant state. Conversely, auxiliary regression can make the same state highly decodable without organizing it in the part of the representation that contributes most strongly to Euclidean distance. Consistent with this interpretation, SCALE also substantially increases held-out latent-state distance alignment, whereas Aux leaves it near the LeWM level despite strong state decodability (Appendix B.1).

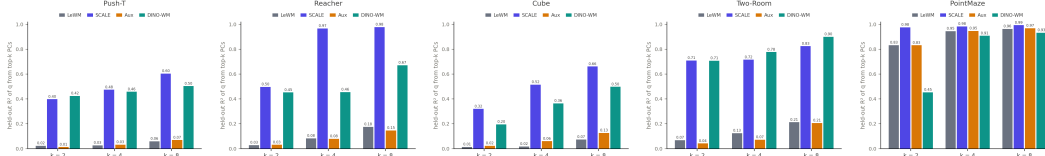


Figure 2: Held-out R^2 for predicting the selected task-relevant state from the leading k principal components of each frozen embedding. SCALE makes the dominant latent subspace more informative about $\mathbf{q}$ than LeWM on every task, and substantially more informative than Aux on Push-T, Reacher, Two Room and Cube despite Aux’s strong full-embedding decodability.

The reorganized geometry survives rollout and improves candidate ranking. The preceding analysis concerns encoded observations, whereas planning operates on predicted future embeddings. For each start and candidate action sequence m , the world model predicts a terminal embedding $\hat{\mathbf{z}}_H^{(m)}$. Executing the same sequence from the same start in the simulator produces $\mathbf{o}_H^{(m),\text{true}}$, which the same encoder maps to $\mathbf{z}_H^{(m),\text{true}} = E_\theta(\mathbf{o}_H^{(m),\text{true}})$.

These paired outcomes define two measurements. First, rollout error compares the predicted and simulator-derived embeddings as $\|\hat{\mathbf{z}}_H^{(m)} - \mathbf{z}_H^{(m),\text{true}}\|_2^2/s$, where s is the model’s mean squared distance between random pairs of the simulator-derived terminal embeddings $\mathbf{z}_H^{(m),\text{true}}$. Second, for the same goal embedding $\mathbf{z}_g$, we compute

$$\hat{J}_m = \|\hat{\mathbf{z}}_H^{(m)} - \mathbf{z}_g\|_2^2, \quad J_m^{\text{true}} = \|\mathbf{z}_H^{(m),\text{true}} - \mathbf{z}_g\|_2^2,$$

and report Kendall’s τ between the candidate rankings induced by $\{\hat{J}_m\}$ and $\{J_m^{\text{true}}\}$.

Table 3: Within-model rollout quality. Lower rollout error and higher Kendall τ are better. Parentheses report change relative to LeWM; when LeWM’s τ is near zero, we report the absolute difference Δ . Results average 20 starts.

Task	Metric	LeWM	SCALE	Aux	DINO-WM
Push-T	RollErr $\downarrow$	.0350	.0277 (−21.0%)	.0269 (−23.1%)	.2819 (+704.9%)
	Kendall $\tau \uparrow$	.7316	.7857 (+7.4%)	.7576 (+3.6%)	.7000 (−4.3%)
Reacher	RollErr $\downarrow$	.1812	.1548 (−14.6%)	.1686 (−7.0%)	.5860 (+223.4%)
	Kendall $\tau \uparrow$	.6233	.6435 (+3.2%)	.6328 (+1.5%)	.6414 (+2.9%)
Cube	RollErr $\downarrow$	.3195	.3174 (−0.7%)	.2389 (−25.2%)	.3227 (+1.0%)
	Kendall $\tau \uparrow$	.3654	.4433 (+21.3%)	.4150 (+13.6%)	.7775 (+112.8%)
Two-Room	RollErr $\downarrow$	.3600	.2857 (−20.6%)	.3511 (−2.5%)	.1479 (−58.9%)
	Kendall $\tau \uparrow$	.5448	.6210 (+14.0%)	.5389 (−1.1%)	.7250 (+33.1%)
PointMaze	RollErr $\downarrow$	2.4226	2.1132 (−12.8%)	2.2418 (−7.5%)	1.9545 (−19.3%)
	Kendall $\tau \uparrow$	−.0029	.1675 (n/a, Δ +.170)	.0540 (n/a, Δ +.057)	.2850 (n/a, Δ +.288)

SCALE increases Kendall ranking agreement over LeWM on every task. The improvement is modest on Push-T and Reacher, where LeWM already preserves candidate ordering relatively well, but becomes much larger on Cube, Two-Room, and PointMaze. This shows that the representational change identified by the PCA analyses is not confined to static encodings: it survives prediction strongly enough to improve the ordering of imagined futures presented to the planner.

The comparison with Aux further separates this effect from generic prediction accuracy. Aux achieves the lowest rollout error on Push-T and Cube, yet its gains in Kendall agreement are consistently smaller than SCALE’s and nearly vanish on the navigation tasks. DINO-WM illustrates the complementary limitation: on Two-Room and PointMaze it combines favorable rollout and ranking behavior, while on Push-T and Reacher its rollout errors are dramatically larger. On Cube, DINO-WM attains the highest Kendall τ without the strongest planning performance. Thus, preserving

a useful candidate ordering is an important interface between representation geometry and planning, but it is not by itself sufficient for planning success; the latent dynamics must also remain predictable.

7 Conclusion

We study how the geometry of a learned representation affects latent-space planning. Our analysis shows that task-relevant information may be readily decodable from an embedding while remaining geometrically weak. Motivated by the contrasting geometries of DINO-WM and LeWM, we introduce SCALE to directly calibrate latent distances with task-relevant state differences during training. The resulting representations preserve LeWM’s end-to-end learning framework while making task-relevant variation more prominent in the geometry seen by the planner. More broadly, our results highlight that, when planning is performed directly with Euclidean latent distance, representation quality depends not only on what information is encoded, but also on how that information is organized geometrically.

References

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023.
- Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.
- Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134(1):19–67, 2005. doi: 10.1007/s10479-005-5724-z.
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with jepa world models, 2025. URL <https://arxiv.org/abs/2601.00844>.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4RL: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.

- Yilun Kuang, Yash Dagade, Quentin Le Lidec, Lucas Maes, Randall Balestriero, and Yann LeCun. LpWM: A case for sparse representations in world models. *arXiv preprint arXiv:2608.22764*, 2026.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, and Sergey Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation*, 2018.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned reinforcement learning. In *International Conference on Learning Representations*, 2025.
- Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Jörg Stückler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1049–1065. PMLR, 2021.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
- Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017. doi: 10.1109/ICRA.2017.7989202.
- Haodong Yan, Jiaguan Zhu, Mingyuan Jia, Ruiqing Yin, Junjie He, Zhide Zhong, Junfeng Li, Jinxuan Lu, Hengtao Li, Tianran Zhang, Jiayi Chen, Wenxuan Song, Wen Chen, Yuxiang Gao, and Haoang Li. Is forward prediction enough? physical state grounding for jepa world models. *arXiv preprint arXiv:2608.06799*, 2026.
- Xi Zeng, Haojie Ren, and Ziyang Song. Phylatent: Learning dynamics-relevant representations for jepa world models. *arXiv preprint arXiv:2608.05720*, 2026.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A Implementation details

A.1 SCALE training pseudocode

Algorithm 1 SCALE training.

Require: $B = (O, A, Q, e)$: aligned pixels O and states Q over $B \times T$ frames, actions A over $B \times (T - 1)$ transitions, and episode IDs $e = (e_b)_{b=1}^B$

Require: $E_\theta, P_\psi, K, \varepsilon, \delta > 0$, and $\lambda_{\text{sig}}, \lambda_{\text{corr}} \geq 0$

- 1: $Z \leftarrow E_\theta(O)$; $\hat{Z} \leftarrow P_\psi(Z_{:,1:T-1}, A)$
- 2: $\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{Z}, Z_{:,2:T})$
- 3: $z \leftarrow \text{reshape}(Z, (N, D))$; $q \leftarrow \text{detach}(\text{reshape}(Q, (N, d_q)))$ $\triangleright N = BT$ frames
- 4: $b_n \leftarrow \lceil n/T \rceil$; $e_n \leftarrow e_{b_n}$ for $n = 1, \dots, N$
- 5: Sample $K/2$ pairs within sub-trajectories ($b_i = b_j$) and $K/2$ across episodes ($e_i \neq e_j$); denote their union by $\mathcal{P}$
- 6: $x_k \leftarrow \|z_{i_k} - z_{j_k}\|_2^2$; $y_k \leftarrow \|q_{i_k} - q_{j_k}\|_2^2$ for $(i_k, j_k) \in \mathcal{P}$
- 7: **if** $\text{std}(x) < \delta$ **or** $\text{std}(y) < \delta$ **then**
- 8: $\mathcal{L}_{\text{corr}} \leftarrow 0 \cdot \text{sum}(z)$ $\triangleright$ graph-connected zero
- 9: **else**
- 10: $\tilde{x} \leftarrow (x - \text{mean}(x)) / (\text{std}(x) + \varepsilon)$; $\tilde{y} \leftarrow (y - \text{mean}(y)) / (\text{std}(y) + \varepsilon)$
- 11: $\mathcal{L}_{\text{corr}} \leftarrow 1 - \text{mean}(\tilde{x} \odot \tilde{y})$
- 12: **end if**
- 13: $\mathcal{L}_{\text{SCALE}} \leftarrow \mathcal{L}_{\text{pred}} + \lambda_{\text{sig}} \text{SIGReg}(Z) + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}}$
- 14: $g_\theta \leftarrow \nabla_\theta \mathcal{L}_{\text{SCALE}}$; $g_\psi \leftarrow \nabla_\psi \mathcal{L}_{\text{pred}}$
- 15: $(\theta, \psi) \leftarrow \text{Adam}((\theta, \psi), (g_\theta, g_\psi))$

Hyperparameters. The encoder is a ViT-Tiny with patch size 14, 12 layers, 3 attention heads, and width 192. Its CLS token is passed through a linear-BatchNorm projector. The predictor is a six-layer causal transformer with 16 attention heads, dropout 0.1, a history length of 3, and actions injected through zero-initialized AdaLN; its output uses a matching projector. We use frame skip 5 and concatenate the five intervening actions into 10-dimensional action blocks. Training uses four-frame sub-trajectories, batch size 128, 10 epochs, and seed 3072. We optimize with Adam using learning rate 5×10^{-5} and weight decay 10^{-3} . SIGReg uses $M=1024$ random unit projections with $\lambda_{\text{sig}} = 0.09$. For $\mathcal{L}_{\text{corr}}$, we sample $K=4096$ pairs per training step with equal within- and cross-episode proportions and set $\varepsilon = 10^{-6}$. We use $\lambda_{\text{corr}} = 0.1$, except for Reacher where it is 0.15. The selected state dimensions for Push-T, Reacher, Cube, Two-Room, and PointMaze are 6, 2, 5, 2, and 2, respectively.

Planner. All planning solvers use a horizon of five action blocks (25 environment steps), an action-block size of five, a receding-horizon interval of five, a 50-step evaluation horizon, and goals sampled 25 steps ahead. The five compute tiers use settings 300/30, 100/20, 50/10, 20/5, and 10/3, corresponding to 9000, 2000, 500, 100, and 30 rollout evaluations per replanning step. The elite count is $\max(\text{round}(0.1 \cdot \text{candidates}), 2)$.

MPPI temperature. MPPI weights candidates by $\exp(-J/T)$ and is therefore sensitive to the scale of J , which differs across representations. For the three LeWM-family models we select a single per-task temperature T^* on a separate tuning seed and report results on five held-out evaluation sets; the selected values are $T^* = 64$ for Push-T and Two-Room, 32 for Reacher and Cube, and 256 for PointMaze, shared by LeWM, SCALE, and Aux so that the comparison among them remains matched. DINO-WM uses $T = 0.5$, which lies in the smooth operating region of its own cost scale.

Paired evaluation. For CEM and iCEM, each of six evaluation seeds defines a set of 100 episodes sampled without replacement, and every training configuration replays the same 600 episodes for each solver and compute tier. Planner randomness is initialized from a hash of the episode identifier and tier that is shared across training configurations, so paired outcomes differ only through the learned models. Standard deviations and standard errors across evaluation sets quantify episode-sampling variation only.

B Additional representation and state-selection results

Tables 4–8 report held-out R^2 for probes from the full frozen embedding to the task-relevant state variables. On Push-T, Two-Room, and PointMaze the probe targets coincide with the selected state $\mathbf{q}$ used for supervision. On Reacher and Cube the probe targets are deliberately broader than $\mathbf{q}$, so that recovery of supervised and unsupervised variables can be compared within the same representation; the supervised subset is identified in each caption. Linear denotes closed-form ridge regression. The MLP is a two-layer network with hidden width 256 and ReLU, trained for 30 epochs using Adam with learning rate 10^{-3} , batch size 1024, and seed 0. Its input contains all principal-component coordinates—an information-preserving rotation of the full embedding—divided by one global scalar; this gives 1499 inputs for DINO-WM and approximately 192 for the LeWM-family models. The finite-optimization MLP generally has lower absolute R^2 than the closed-form probe, so comparisons are made within each probe family. Aux denotes the latent-to-state regression control of Sec. 5.2.

Table 4: Per-component held-out R^2 for Push-T.

Probe	State component	LeWM	SCALE	Aux	DINO-WM
Linear	agent x	.9191	.9656	.9851	.9342
	agent y	.9194	.9715	.9845	.9394
	block x	.9686	.9820	.9904	.9291
	block y	.9603	.9746	.9886	.9722
	$\cos \theta$	.9019	.9023	.9767	.8353
	$\sin \theta$	.8694	.8855	.9778	.5943
	Mean	.9231	.9469	.9839	.8674
MLP	agent x	.8580	.8876	.9187	.7704
	agent y	.8676	.9206	.9066	.7676
	block x	.9219	.9005	.9429	.7847
	block y	.9149	.8943	.9475	.7685
	$\cos \theta$	.8572	.8569	.8980	.5792
	$\sin \theta$	.8404	.8605	.9116	.4869
	Mean	.8767	.8867	.9209	.6929

Table 5: Per-component held-out R^2 for Reacher. The selected state $\mathbf{q}$ comprises $\cos q_0$ and $\sin q_0$; $\cos q_1$ and $\sin q_1$ are probed but never supervised.

Probe	State component	LeWM	SCALE	Aux	DINO-WM
Linear	$\cos q_0$	.9996	.9998	.9996	.9972
	$\sin q_0$	.9997	.9998	.9998	.9981
	$\cos q_1$	.9994	.9997	.9995	.9953
	$\sin q_1$	.9980	.9996	.9986	.9702
	Mean	.9992	.9997	.9994	.9902
MLP	$\cos q_0$	.9837	.9875	.9854	.9479
	$\sin q_0$	.9835	.9882	.9846	.9245
	$\cos q_1$	.9780	.9800	.9820	.8612
	$\sin q_1$	.9838	.9886	.9830	.7814
	Mean	.9823	.9861	.9838	.8788

Table 6: Per-component held-out R^2 for Cube. The selected state $\mathbf{q}$ comprises effector x, y, z and $\cos 2\psi, \sin 2\psi$; gripper and block x, y, z are probed but never supervised.

Probe	State component	LeWM	SCALE	Aux	DINO-WM
Linear	effector x	.9773	.9854	.9934	.9814
	effector y	.9661	.9778	.9943	.9980
	effector z	.9825	.9899	.9928	.9818
	$\cos 2\psi$	.7599	.9888	.9938	.7627
	$\sin 2\psi$	-.2577	.9880	.9938	.4213
	gripper	.9072	.9783	.9742	.6638
	block x	.9870	.9922	.9950	.9282
	block y	.9884	.9792	.9924	.9804
	block z	.9869	.9935	.9937	.8871
	Mean	.8109	.9859	.9915	.8450
MLP	effector x	.8781	.8326	.8630	.7686
	effector y	.9038	.9059	.9200	.8992
	effector z	.8420	.8603	.8111	.8106
	$\cos 2\psi$	.6599	.9034	.9091	.5339
	$\sin 2\psi$	-.0394	.8868	.8724	.2822
	gripper	.7974	.8871	.8627	.4786
	block x	.9083	.8537	.9145	.7291
	block y	.9324	.9081	.9123	.8928
	block z	.8846	.9024	.9086	.5871
	Mean	.7519	.8823	.8860	.6647

Table 7: Per-component held-out R^2 for Two-Room.

Probe	State component	LeWM	SCALE	Aux	DINO-WM
Linear	x	.9957	.9982	.9963	.9957
	y	.9942	.9977	.9981	.9975
	Mean	.9950	.9979	.9972	.9966
MLP	x	.9581	.9742	.9722	.9315
	y	.9660	.9829	.9714	.9209
	Mean	.9621	.9786	.9718	.9262

Table 8: Per-component held-out R^2 for PointMaze.

Probe	State component	LeWM	SCALE	Aux	DINO-WM
Linear	x	.9988	.9992	.9989	.9982
	y	.9989	.9990	.9989	.9982
	Mean	.9989	.9991	.9989	.9982
MLP	x	.9703	.9869	.9717	.9288
	y	.9824	.9733	.9796	.9609
	Mean	.9764	.9801	.9757	.9449

State selection. On Reacher and Cube we supervise a strict subset of the available simulator state. Enlarging the selection does not help: average success moves from 66.17 to 65.75 on Reacher when both joint angles are supervised rather than the first alone, and from 62.65 to 62.55 on Cube when the gripper and block coordinates are added to the five selected variables. Reducing the six-dimensional Push-T state does not help either. Under the account of Sec. 4.2 this is expected rather than surprising: $\mathcal{L}_{\text{corr}}$ distributes metric leverage over whatever variation $\mathbf{q}$ contains, so adding variables that matter less for the task dilutes the leverage available to those that matter more. State selection therefore acts as a task-dependent inductive bias, and more supervised variables are not automatically better.

B.1 Held-out latent–state distance alignment

As a direct held-out check of the geometric quantity optimized by SCALE, we measure the Spearman rank correlation between pairwise latent distances and task-state distances using the protocol of Sec. 4.1. Because this quantity is directly targeted by $\mathcal{L}_{\text{corr}}$, we treat it as a consistency check rather than as independent evidence for the mechanism developed in Sec. 6.3.

Table 9: Held-out latent–state distance rank alignment (Spearman ρ) over 1.12M frame pairs per task. LeWM and DINO-WM repeat Table 1 for comparison.

Task	LeWM	SCALE	Aux	DINO-WM
Push-T	.13	.77	.18	.70
Reacher	.18	.53	.17	.46
Cube	.00	.56	.03	.46
Two-Room	.41	.77	.41	.85
PointMaze	.52	.80	.52	.50

SCALE substantially increases latent–state rank alignment over LeWM on every task. Aux, in contrast, leaves the alignment essentially unchanged despite making task state highly recoverable from the full embedding. The latter provides an additional check on the distinction motivating SCALE: state decodability alone does not ensure that state differences are expressed in the latent metric.

C Full planning results

Tables 10–14 report success rates for every task, planning solver, and compute tier, including DINO-WM as the pretrained-feature reference. CEM and iCEM entries average six paired evaluation sets; MPPI entries use the per-task temperature protocol of Appendix A and average five held-out evaluation sets.

Table 10: Push-T success rates (%) for all solvers. Mean $\pm$ SD per tier; the final column averages T1–T5. Bold marks the best method per column within each solver.

Solver	Method	T1	T2	T3	T4	T5	Mean
CEM	LeWM	92.3 $\pm$ 2.0	90.5 $\pm$ 3.3	79.2 $\pm$ 5.0	57.0 $\pm$ 7.1	33.5 $\pm$ 2.7	70.5
	SCALE	96.0$\pm$2.0	93.2 $\pm$ 2.6	84.7$\pm$4.0	63.0 $\pm$ 4.6	32.5 $\pm$ 4.4	73.9
	Aux.	94.2 $\pm$ 1.9	93.3$\pm$2.0	84.7$\pm$5.3	66.2$\pm$8.9	36.2$\pm$7.9	74.9
	DINO-WM	73.8 $\pm$ 4.2	72.5 $\pm$ 4.9	65.0 $\pm$ 5.8	46.8 $\pm$ 5.0	24.8 $\pm$ 5.6	56.6
iCEM	LeWM	86.2 $\pm$ 3.3	80.7 $\pm$ 2.2	73.5 $\pm$ 6.5	53.5 $\pm$ 5.7	25.7 $\pm$ 5.4	63.9
	SCALE	89.7 $\pm$ 2.0	87.0 $\pm$ 2.5	79.0 $\pm$ 3.9	59.8 $\pm$ 4.7	25.5 $\pm$ 5.8	68.2
	Aux.	91.3$\pm$1.2	87.7$\pm$1.8	80.3$\pm$4.0	60.5$\pm$6.0	26.7$\pm$4.6	69.3
	DINO-WM	70.5 $\pm$ 4.2	65.2 $\pm$ 4.2	58.5 $\pm$ 5.2	39.8 $\pm$ 10.0	22.5 $\pm$ 4.3	51.3
MPPI	LeWM	80.2 $\pm$ 2.6	82.6 $\pm$ 1.1	75.0 $\pm$ 3.6	58.4 $\pm$ 3.4	35.8 $\pm$ 5.2	66.4
	SCALE	88.6$\pm$3.8	87.8 $\pm$ 2.8	81.0$\pm$3.7	59.4$\pm$4.2	35.4 $\pm$ 4.0	70.4
	Aux.	87.8 $\pm$ 4.3	89.2$\pm$3.3	80.6 $\pm$ 4.0	58.8 $\pm$ 4.9	37.8$\pm$5.8	70.8
	DINO-WM	76.4 $\pm$ 3.8	72.6 $\pm$ 3.8	63.2 $\pm$ 5.8	47.4 $\pm$ 8.8	31.2 $\pm$ 6.2	58.2

Table 11: Reacher success rates (%) for all solvers. Mean $\pm$ SD per tier; the final column averages T1–T5. Bold marks the best method per column within each solver.

Solver	Method	T1	T2	T3	T4	T5	Mean
CEM	LeWM	81.8 $\pm$ 1.2	78.8$\pm$4.4	72.7 $\pm$ 4.3	64.3 $\pm$ 4.9	55.2 $\pm$ 5.8	70.6
	SCALE	82.5$\pm$4.6	78.7 $\pm$ 3.8	74.8$\pm$5.9	64.5 $\pm$ 8.4	57.2$\pm$6.1	71.5
	Aux.	80.7 $\pm$ 4.9	77.3 $\pm$ 4.5	74.2 $\pm$ 3.7	65.0$\pm$5.0	55.0 $\pm$ 7.7	70.4
	DINO-WM	76.8 $\pm$ 3.5	74.0 $\pm$ 5.8	70.8 $\pm$ 4.2	61.5 $\pm$ 4.0	47.7 $\pm$ 7.3	66.2
iCEM	LeWM	85.5 $\pm$ 2.4	81.8 $\pm$ 3.1	83.3 $\pm$ 3.5	77.8 $\pm$ 6.1	61.7 $\pm$ 2.9	78.0
	SCALE	86.7$\pm$3.6	86.2 $\pm$ 2.9	86.5 $\pm$ 3.4	82.3$\pm$3.2	64.0$\pm$2.5	81.1
	Aux.	84.3 $\pm$ 4.6	87.0$\pm$2.8	86.8$\pm$3.5	81.2 $\pm$ 3.8	61.5 $\pm$ 4.1	80.2
	DINO-WM	79.0 $\pm$ 4.6	77.5 $\pm$ 5.6	79.7 $\pm$ 3.1	75.5 $\pm$ 5.0	52.3 $\pm$ 6.0	72.8
MPPI	LeWM	56.6 $\pm$ 6.2	58.2 $\pm$ 1.9	52.4 $\pm$ 5.0	55.0$\pm$6.1	50.4 $\pm$ 4.4	54.5
	SCALE	57.8 $\pm$ 3.8	62.2$\pm$3.6	56.4 $\pm$ 2.9	54.6 $\pm$ 3.4	54.4$\pm$2.9	57.1
	Aux.	51.0 $\pm$ 2.9	56.0 $\pm$ 6.6	56.6$\pm$4.4	51.4 $\pm$ 6.3	49.0 $\pm$ 5.5	52.8
	DINO-WM	59.4$\pm$3.2	60.0 $\pm$ 6.6	54.6 $\pm$ 3.0	50.2 $\pm$ 8.5	48.6 $\pm$ 8.4	54.6

Table 12: Cube success rates (%) for all solvers. Mean $\pm$ SD per tier; the final column averages T1–T5. Bold marks the best method per column within each solver.

Solver	Method	T1	T2	T3	T4	T5	Mean
CEM	LeWM	67.5 $\pm$ 5.9	67.0 $\pm$ 3.8	61.2 $\pm$ 5.8	57.3$\pm$3.6	53.8 $\pm$ 5.0	61.4
	SCALE	73.5$\pm$3.4	71.3$\pm$4.2	65.5$\pm$4.8	56.3 $\pm$ 5.2	54.2 $\pm$ 5.0	64.2
	Aux.	73.5$\pm$5.0	70.3 $\pm$ 4.6	62.8 $\pm$ 4.9	55.5 $\pm$ 5.2	55.3$\pm$4.2	63.5
	DINO-WM	72.5 $\pm$ 2.4	68.0 $\pm$ 1.3	58.8 $\pm$ 2.5	52.3 $\pm$ 4.3	54.5 $\pm$ 4.1	61.2
iCEM	LeWM	70.7 $\pm$ 5.3	68.0 $\pm$ 3.9	69.7 $\pm$ 3.6	62.7 $\pm$ 4.7	55.2 $\pm$ 5.7	65.3
	SCALE	76.7$\pm$3.9	76.5$\pm$4.5	74.8$\pm$3.5	64.8$\pm$3.7	59.7$\pm$4.5	70.5
	Aux.	74.5 $\pm$ 3.2	73.3 $\pm$ 4.5	73.0 $\pm$ 3.1	64.5 $\pm$ 3.3	57.7 $\pm$ 3.8	68.6
	DINO-WM	70.8 $\pm$ 2.6	67.3 $\pm$ 1.5	66.2 $\pm$ 1.7	57.3 $\pm$ 4.5	51.7 $\pm$ 3.8	62.7
MPPI	LeWM	58.6 $\pm$ 3.6	60.2 $\pm$ 5.2	54.4 $\pm$ 4.8	54.2 $\pm$ 5.3	54.6$\pm$6.3	56.4
	SCALE	64.4$\pm$4.4	63.0 $\pm$ 3.5	57.2$\pm$4.1	54.6$\pm$2.1	52.6 $\pm$ 4.9	58.4
	Aux.	64.2 $\pm$ 5.9	64.0$\pm$4.0	54.6 $\pm$ 5.0	53.6 $\pm$ 6.1	53.4 $\pm$ 7.3	58.0
	DINO-WM	64.4$\pm$4.0	62.6 $\pm$ 3.4	52.6 $\pm$ 3.2	50.8 $\pm$ 5.0	53.0 $\pm$ 5.2	56.7

Table 13: Two-Room success rates (%) for all solvers. Mean $\pm$ SD per tier; the final column averages T1–T5. Bold marks the best method per column within each solver.

Solver	Method	T1	T2	T3	T4	T5	Mean
CEM	LeWM	88.0 $\pm$ 2.0	87.7 $\pm$ 2.6	84.3 $\pm$ 3.4	71.7 $\pm$ 3.5	58.0 $\pm$ 4.4	77.9
	SCALE	99.8 $\pm$ 0.4	99.3 $\pm$ 0.8	93.7 $\pm$ 2.5	82.3 $\pm$ 3.0	71.0 $\pm$ 3.3	89.2
	Aux.	87.8 $\pm$ 3.2	87.5 $\pm$ 3.6	82.7 $\pm$ 3.4	72.0 $\pm$ 2.7	60.5 $\pm$ 2.0	78.1
	DINO-WM	100.0$\pm$0.0	99.8$\pm$0.4	98.5$\pm$1.4	93.8$\pm$2.3	87.2$\pm$3.7	95.9
iCEM	LeWM	91.3 $\pm$ 1.2	93.3 $\pm$ 1.2	89.2 $\pm$ 2.5	77.5 $\pm$ 1.4	63.2 $\pm$ 3.0	82.9
	SCALE	100.0$\pm$0.0	99.8 $\pm$ 0.4	97.3 $\pm$ 1.4	90.5 $\pm$ 1.4	76.7 $\pm$ 1.9	92.9
	Aux.	92.5 $\pm$ 2.7	93.5 $\pm$ 2.1	88.5 $\pm$ 1.9	79.0 $\pm$ 2.4	62.0 $\pm$ 1.8	83.1
	DINO-WM	100.0$\pm$0.0	100.0$\pm$0.0	99.7$\pm$0.5	98.7$\pm$0.5	92.8$\pm$1.7	98.2
MPPI	LeWM	91.8 $\pm$ 1.8	91.4 $\pm$ 2.4	86.0 $\pm$ 4.4	74.8 $\pm$ 2.3	68.0 $\pm$ 3.4	82.4
	SCALE	95.2$\pm$2.4	97.2 $\pm$ 2.5	91.4 $\pm$ 1.7	86.2 $\pm$ 2.2	78.0 $\pm$ 3.7	89.6
	Aux.	93.2 $\pm$ 1.5	90.4 $\pm$ 1.5	86.2 $\pm$ 3.8	76.6 $\pm$ 4.0	67.6 $\pm$ 1.3	82.8
	DINO-WM	93.8 $\pm$ 1.8	98.4$\pm$1.5	93.8$\pm$2.5	88.0$\pm$2.3	90.8$\pm$1.3	93.0

Table 14: PointMaze success rates (%) for all solvers. Mean $\pm$ SD per tier; the final column averages T1–T5. Bold marks the best method per column within each solver.

Solver	Method	T1	T2	T3	T4	T5	Mean
CEM	LeWM	83.0 $\pm$ 5.0	83.7 $\pm$ 3.9	85.3 $\pm$ 5.0	83.3 $\pm$ 3.9	75.3 $\pm$ 7.0	82.1
	SCALE	89.2 $\pm$ 3.7	87.3 $\pm$ 5.1	87.7 $\pm$ 3.3	86.3 $\pm$ 2.3	80.5 $\pm$ 1.9	86.2
	Aux.	88.2 $\pm$ 3.4	87.0 $\pm$ 4.0	84.7 $\pm$ 4.6	80.3 $\pm$ 4.5	81.2 $\pm$ 5.7	84.3
	DINO-WM	97.7$\pm$0.8	98.0$\pm$1.3	97.7$\pm$1.5	93.5$\pm$2.8	87.5$\pm$3.3	94.9
iCEM	LeWM	81.7 $\pm$ 3.9	82.7 $\pm$ 4.4	81.3 $\pm$ 3.6	81.7 $\pm$ 5.1	73.7 $\pm$ 4.7	80.2
	SCALE	87.3 $\pm$ 3.9	82.7 $\pm$ 3.9	84.3 $\pm$ 4.1	83.2 $\pm$ 4.6	79.3 $\pm$ 2.4	83.4
	Aux.	83.8 $\pm$ 4.8	82.8 $\pm$ 3.7	80.5 $\pm$ 5.5	81.2 $\pm$ 2.9	78.3 $\pm$ 5.5	81.3
	DINO-WM	94.2$\pm$1.7	93.7$\pm$2.3	92.8$\pm$2.3	90.8$\pm$1.7	86.2$\pm$1.9	91.5
MPPI	LeWM	83.2 $\pm$ 5.1	83.4 $\pm$ 4.6	84.2 $\pm$ 4.5	85.0 $\pm$ 6.6	75.2 $\pm$ 5.4	82.2
	SCALE	90.2 $\pm$ 2.7	90.0 $\pm$ 2.5	88.4 $\pm$ 2.6	87.8 $\pm$ 2.5	80.2 $\pm$ 6.2	87.3
	Aux.	87.6 $\pm$ 2.8	89.2 $\pm$ 1.8	88.0 $\pm$ 1.6	83.0 $\pm$ 2.7	84.4 $\pm$ 3.8	86.4
	DINO-WM	99.0$\pm$1.0	99.2$\pm$1.3	97.4$\pm$1.9	91.4$\pm$2.7	92.8$\pm$4.1	96.0